INSTITUTO TECNOLÓGICO METROPOLITANO
Programación de Software

Sebastian Rios Rios

Sistema de Inventario para Tienda de Belleza

API REST con Spring Boot y SQL Server

Objetivo General

Desarrollar un sistema de gestión de inventario para una tienda de productos de belleza que permita controlar de manera eficiente el registro, consulta, actualización y eliminación de productos, con el fin de mejorar la organización del negocio y facilitar el control del stock disponible.

Objetivos Específicos

- Permitir el registro de nuevos productos en el inventario incluyendo su nombre, categoría, proveedor, precio y cantidad disponible, para que la tienda tenga visibilidad completa de los artículos que maneja.
- Facilitar la consulta del inventario de forma completa o por producto específico, de manera que el personal pueda acceder rápidamente a la información que necesita.
- Permitir la actualización de los datos de cualquier producto cuando ocurran cambios en precio, stock u otras características, manteniendo la información siempre al día.
- Gestionar las categorías de productos para que el inventario esté organizado por tipo de artículo, facilitando la búsqueda y el control de cada segmento del negocio.
- Administrar la información de los proveedores asociados a cada producto, permitiendo conocer de dónde proviene cada artículo y facilitando la reposición de stock.
- Habilitar la eliminación de productos que ya no estén disponibles, manteniendo el inventario limpio y libre de información desactualizada.
- Apoyar el control del stock mínimo de cada producto, ayudando a identificar cuándo un artículo está próximo a agotarse y evitando quiebres de inventario.

1. Introducción

se desarrolló un servicio web de tipo REST para gestionar el inventario de una tienda de productos de belleza. La aplicación permite realizar operaciones CRUD completas

sobre productos, categorías y proveedores a través de endpoints HTTP.

El modelo de datos contempla tres entidades principales: Producto, Categoría y Proveedor, relacionadas entre sí mediante llaves foráneas en SQL Server. Cada entidad cuenta con su propio conjunto de endpoints documentados en Swagger UI.

2. Tecnologías Utilizadas

Tecnología	Versión	Uso
Java	17	Lenguaje principal
Spring Boot	3.2.0	Framework backend
SQL Server	2019+	Base de datos relacional
mssql-jdbc	12.4.2	Driver de conexión JDBC manual
SpringDoc OpenAPI	2.3.0	Documentación Swagger UI
Maven	Wrapper	Gestión de dependencias y build
Git / GitHub	-	Control de versiones con ramas

3. Arquitectura del Proyecto

El proyecto implementa una arquitectura por capas que separa las responsabilidades de cada componente, facilitando el mantenimiento y la comprensión del código:

```
com.belleza.inventario
■■■ controllers/ → Exponen los endpoints REST (GET, POST, PUT, DELETE)
■■■ services/ → Contienen la lógica de negocio
■■■ dao/ → Patrón DAO: interfaz + implementación con SQL puro
■■■ entities/ → POJOs que representan las tablas de la BD
■■■ util/ → ConexionDB: manejo manual del driver JDBC
```

Se implementaron tres módulos completos siguiendo la misma arquitectura: Producto, Categoria y Proveedor. Cada uno tiene su entidad, interfaz DAO, implementación DAO, servicio y controlador REST.

4. Base de Datos

Se utilizó SQL Server como motor de base de datos relacional. La base de datos inventario_belleza contiene siete tablas relacionadas entre sí mediante llaves foráneas.

4.1 Tabla: producto

Campo	Tipo	Descripción	Restricción
id_producto	INT	Identificador único	PK, IDENTITY
nombre	VARCHAR(100)	Nombre del producto	NOT NULL
descripcion	VARCHAR(255)	Descripción del artículo	-
precio	DECIMAL(10,2)	Precio unitario	NOT NULL
stock	INT	Cantidad disponible	NOT NULL, DEFAULT 0
stock_minimo	INT	Cantidad mínima alerta	DEFAULT 5
id_categoria	INT	Referencia a categoría	FK → categoria
id_proveedor	INT	Referencia a proveedor	FK → proveedor

4.2 Tabla: categoria

Campo	Tipo	Descripción	Restricción
id_categoria	INT	Identificador único	PK, IDENTITY
nombre	VARCHAR(100)	Nombre de la categoría	NOT NULL

4.3 Tabla: proveedor

Campo	Tipo	Descripción	Restricción
id_proveedor	INT	Identificador único	PK, IDENTITY
nombre	VARCHAR(100)	Nombre del proveedor	NOT NULL
telefono	VARCHAR(20)	Teléfono de contacto	-
email	VARCHAR(100)	Correo electrónico	-

4.4 Script SQL principal

```
CREATE DATABASE inventario_belleza;
USE inventario_belleza;

CREATE TABLE categoria (
id_categoria INT PRIMARY KEY IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL
);

CREATE TABLE proveedor (
id_proveedor INT PRIMARY KEY IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL,
```

```
telefono VARCHAR(20), email VARCHAR(100)
);

CREATE TABLE producto (
id_producto INT PRIMARY KEY IDENTITY(1,1),
nombre VARCHAR(100) NOT NULL,
descripcion VARCHAR(255),
precio DECIMAL(10,2) NOT NULL,
stock INT NOT NULL DEFAULT 0,
stock_minimo INT DEFAULT 5,
id_categoria INT REFERENCES categoria(id_categoria),
id_proveedor INT REFERENCES proveedor(id_proveedor)
);
```

5. Explicación de cada Capa

5.1 Entidades

Las entidades son clases Java que representan las tablas de la base de datos. Cada entidad contiene los atributos correspondientes a las columnas de su tabla, un constructor vacío requerido por Spring Boot, y los métodos getters y setters para acceder a cada atributo. Se implementaron tres entidades: Producto, Categoria y Proveedor.

5.2 Capa DAO

Es la capa de persistencia donde se maneja todo lo relacionado con la base de datos. Se implementó el patrón DAO con interfaz e implementación para cada entidad:

- **ConexionDB.java:** Maneja la conexión manual a SQL Server usando DriverManager.getConnection() sin pool de conexiones.
- **ProductoDAO.java:** Interfaz que define los métodos: obtenerTodos(), obtenerPorId(), crear(), actualizar() y eliminar().
- **ProductoDAOImpl.java:** Implementación con consultas SQL escritas manualmente usando PreparedStatement, sin ORM.

Ejemplo de consulta SQL en ProductoDAOImpl:

```
String sql = "SELECT id_producto, nombre, descripcion, precio,
stock, stock_minimo, id_categoria, id_proveedor
FROM producto WHERE id_producto = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setInt(1, id);
```

5.3 Capa Service

Contiene la lógica de negocio y actúa como puente entre el controlador y el DAO. Cada servicio está anotado con @Service y delega las operaciones al DAO correspondiente. Se implementaron: ProductoService, CategoriaService y ProveedorService.

5.4 Capa Controller

Expone los endpoints REST al exterior. Cada controlador está anotado con @RestController y @RequestMapping. Se implementaron tres controladores completos:

Método	URL	Acción
GET	/api/productos	Listar todos los productos
GET	/api/productos/{id}	Buscar producto por ID
POST	/api/productos	Crear nuevo producto
PUT	/api/productos/{id}	Actualizar producto
DELETE	/api/productos/{id}	Eliminar producto
GET	/api/categorias	Listar todas las categorías
GET	/api/categorias/{id}	Buscar categoría por ID
POST	/api/categorias	Crear nueva categoría
PUT	/api/categorias/{id}	Actualizar categoría
DELETE	/api/categorias/{id}	Eliminar categoría
GET	/api/proveedores	Listar todos los proveedores
GET	/api/proveedores/{id}	Buscar proveedor por ID
POST	/api/proveedores	Crear nuevo proveedor
PUT	/api/proveedores/{id}	Actualizar proveedor
DELETE	/api/proveedores/{id}	Eliminar proveedor

6. Documentación con Swagger

Para documentar el servicio web se utilizó SpringDoc OpenAPI 2.3.0. Se configuró la ruta en application.properties y se anotaron los endpoints con @Tag, @Operation y @ApiResponse. La documentación interactiva queda disponible en:

```
http://localhost:8080/api/swagger-ui.html
```

Desde Swagger UI se pueden visualizar los 15 endpoints disponibles agrupados en tres secciones (Productos, Categorias, Proveedores) y probarlos directamente desde el navegador sin herramientas adicionales.

7. Configuración del Proyecto

Archivo application.properties:

```
spring.application.name=inventario
server.port=8080
server.servlet.context-path=/api

spring.datasource.url=jdbc:sqlserver://localhost:1433;
databaseName=inventario belleza;encrypt=false;integratedSecurity=tr
ue
spring.datasource.driver-class-
name=com.microsoft.sqlserver.jdbc.SQLServerDriver

springdoc.swagger-ui.path=/swagger-ui.html
```

Se utilizó autenticación integrada de Windows (integratedSecurity=true) para la conexión a SQL Server, por lo que no fue necesario configurar usuario y contraseña de forma explícita.

8. Control de Versiones con Git y GitHub

El proyecto se gestionó en un repositorio de GitHub con una estrategia de ramas por funcionalidad. Se creó una rama independiente para cada módulo implementado y se integró a main mediante Pull Requests:

Rama	Descripción	Estado
main	Rama principal del proyecto	Activa
feature/producto	CRUD completo de Producto con schema SQL real	**Mergeada**
feature/categoria	CRUD completo de Categoria con DAO y SQL puro	**Mergeada**
feature/proveedor	CRUD completo de Proveedor con DAO y SQL puro	**Mergeada**

Historial de commits principal:

```
40d3f71 Merge pull request #3 from sebitasrios/feature/proveedor
ce7d799 Merge pull request #2 from sebitasrios/feature/categoria
ddb875c Merge pull request #1 from sebitasrios/feature/producto
d627a89 feat: CRUD completo de Proveedor con DAO y SQL puro
853ad40 feat: CRUD completo de Categoria con DAO y SQL puro
a78be46 feat: CRUD completo de Producto actualizado con schema SQL
real
```

9. Conclusiones

Con el desarrollo de este proyecto se pusieron en práctica los conceptos fundamentales de los servicios web REST aplicados a un caso de negocio real. Se logró estructurar una API completa con tres módulos (Producto, Categoria y Proveedor), cada uno siguiendo la arquitectura por capas y el patrón DAO, realizando consultas SQL manuales sin depender de ningún ORM.

El modelo relacional de la base de datos quedó alineado con el código Java, resolviendo la inconsistencia que existía entre los nombres de columnas del SQL y los atributos de las entidades. La documentación con Swagger permite probar los 15 endpoints disponibles directamente desde el navegador.

El uso de Git con ramas por funcionalidad permitió llevar un control organizado del desarrollo, con Pull Requests documentados para cada módulo integrado a la rama principal.